

A tabu search procedure based on a random Roulette diversification for the weighted maximal planar graph problem

Ibrahim H. Osman*

Suliman S. Olayan School of Business, American University of Beirut, P.O. Box 11-0236, Beirut, Lebanon

Available online 27 October 2005

Abstract

An efficient and effective tabu search implementation for the weighted maximal planar graph problem is proposed. The search incorporates a number of novel features including: the introduction of a new set of two-move operators; a move-cache-memory structure for efficiently searching the neighbourhood space; and a random roulette selection from a ranked list of best moves for diversification. The effects of these and other features on solution quality are investigated. Computational results are reported on a set of 100 benchmark instances of sizes varying from 20 to 100 vertices. The results demonstrate that the new probabilistic tabu search algorithm is very competitive with state of the art algorithms in the literature in terms of both solution quality and computational effort.

Scope

The weighted maximal planar graph (WMPG) problem seeks to find a sub-graph from a given weighted complete graph such that the subgraph is planar, i.e., it can be embedded on the plane without any edges intersecting. It is maximal, i.e., no additional edge can be added to the sub-graph without destroying its planarity and it has also the maximal sum of edge weights. The WMPG problem has applications in modern manufacturing systems, e.g., arranging rooms within a building floor plan, and placing machines on a factory floor or components on a circuit board. The WMPG problem is NP-Hard. In the absence of a polynomial-time algorithm to solve it exactly, it still attracts research on approximate approaches.

© 2005 Elsevier Ltd. All rights reserved.

Keywords: Combinatorial optimization problem; Data structure; Diversification strategy; Facility layout; Graph theory; Construction heuristics; Meta-heuristics; Tabu search; Weighted maximal planar graph

* Tel.: +961 1 352 700; fax: +961 1 750 214.

E-mail address: ibrahim.osman@aub.edu.lb.

1. Introduction

Let $G = (V, E, W)$ be a complete undirected weighted graph, where V is the set of vertices, E is the set of edges, and W is the set of positive weights associated with the edges. Each edge $k \in E$ is represented by $k = \{n_i, n_j\}$, where n_i and n_j are its two end vertices. It also has a positive weight w_k representing the benefit (or desirability value) derived from the two end vertices being adjacent. A graph is *planar* if it can be drawn on the plane without any edges intersecting. A *planar graph* is *maximal* if no additional edge can be added to it without destroying its planarity. The objective of the weighted maximal planar graph (WMPG) problem is to find a maximal planar sub-graph $G_p = (V, E_p, W_p)$ with the maximal (highest) sum of edge weights, $W(G_p) = \sum_{k \in E_p} w_k$, where $E_p \subseteq E$.

The WMPG problem is of practical importance in modern manufacturing systems. It has many practical applications in arranging rooms within a building floor, placing machines on a factory floor, and locating controls on an instrument panel or components on a circuit board, [1,2]. Applications also include the location of electrical circuits in VLSI design [3]; graph planarization [4]; and automatic graph drawing [5,6].

Applications of the WMPG problem to the design of skeletons for a family of facility layouts and location problems are based on a systematic three-phase approach. *First*, an adjacency graph is constructed in which the vertices represent facilities, the edges define the relationships between the facilities, and the weights on edges express the material flow (or adjacency benefit) between facilities. *Secondly*, a WMPG optimal solution of the constructed adjacency graph is obtained. *Finally*, to satisfy space and shape requirements, facility arrangement, and adjacencies the designer draws a block layout using the dual of the WMPG optimal solution. Fig. 1 illustrates a WMPG solution $G_p = (V, E_p, W_p)$, where the set E_p is represented by solid lines and its corresponding block layout by dashed lines. It can be seen that each vertex of G_p is associated with one facility area including the external area. Each edge of E_p is translated into a common wall (a dashed boundary) between two adjacent facilities in the final block layout. Moreover, all the faces of G_p are triangular, i.e., each is bounded by three edges and determined by three adjacent vertices. For more details on the block layout drawings, we refer to Giffin et al. [7], Hassan and Hogg [8], Al-Hakim [9], Irvine and Rinsam-Melchert [10] and Watson and Giffin [11].

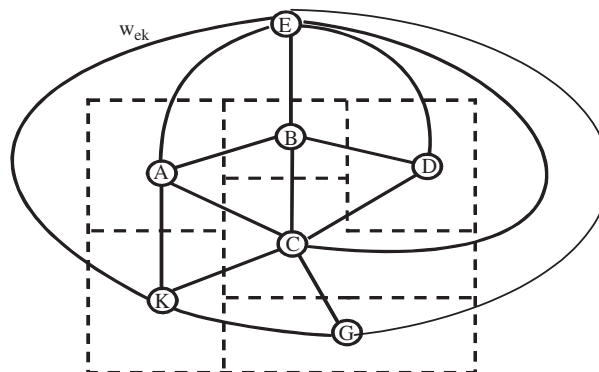


Fig. 1. A WMPG solution (solid lines) and its block layout (dashed lines).

The most difficult part of the systematic three-phase approach is the finding of the WMPG optimal solution. The WMPG problem is an important and challenging combinatorial optimization problem since it is NP-Hard [12]. Thus, WMPG instances of practical sizes can not be efficiently solved to optimality by existing exact approaches and approximate methods are the only practical tool for solving such real-world instances.

Meta-Heuristics are recent approximate techniques that have achieved widespread success in providing high-quality, near-optimal solutions to many real-life complex optimization problems in diverse areas. We refer to Osman [13,14] for an introduction and an overview, to Osman and Laporte [15] for a bibliography and to the books by Osman and Kelly [16], Rayward-Smith et al. [17], Voss et al. [18], Ribeiro and Hansen [19], Glover and Kochenberger [20] and Resende and de Sousa [21].

Tabu search (TS) is one of the most successful meta-heuristic in combinatorial optimization [22,23]. Success of TS motivated us to design and implement an efficient and effective implementation for the WMPG problem. The implementation has a number of new features and contributions. First, a set of two new move operators for generating neighboring solutions is introduced. The first operator is the *split-edge operator*, which is based on inserting a vertex on an edge, and the second is the *split-face operator*, which is based on inserting a vertex into the middle of three consecutive triangular faces. Secondly, a cache-memory data structure is implemented for efficiently searching the neighborhood space. Thirdly, a random roulette selection strategy, based on ranking the best moves in the cache memory, is used for search diversification. The effect of each feature on solution quality is empirically investigated.

The remaining parts of the paper are organized as follows. Section 2 presents a brief literature review of the WMPG and its graph-theoretic properties. The deltahedron heuristic of Foulds and Robinson [24] for constructing an initial starting solution is presented in Section 3. Our TS implementation is explained in Section 4. Computational results are reported using a set of 100 test instances of size varying from 20 to 100 vertices. The efficiency and effectiveness of the proposed approach is demonstrated by comparing it to the best-published algorithms in the literature in Section 5. Finally, we conclude with some remarks in Section 6.

2. Brief review of the WMPG and its properties

Let $G = (V, E, W)$ be a complete graph, where $V = \{1, \dots, n\}$ is the set of n vertices, $E = \{1, \dots, a\}$ is the set of undirected edges, $W = \{w_k | w_k \geq 0, \forall k \in E\}$ is the set of edge weights. Let $G_p = (V, E_p, W_p)$ be a corresponding weighted maximal planar sub-graph solution defined by the set of edges $E_p = \{a_1, \dots, a_p\}$, and the set of triangular faces $F = \{f_1, \dots, f_t\}$ then, Euler's formula ($n - p + t = 2$) and the following implied properties are valid for any $n \geq 3$:

- (i) For any given planar graph, $p \leq 3n - 6$ and $p = 3n - 6$ when it is maximal.
- (ii) If G_p is a bipartite (connected) graph then the number of triangular faces is $t = 2n - 4$.

From the above properties, the combinatorial nature of the weighted maximal planar graph can be illustrated from the need to find a set E_p of $3n - 6$ edges out of the total of number of edges $a = \frac{n(n-1)}{2}$ and a set F of $2n - 4$ triangular faces out the total number of faces $f = \frac{n(n-1)(n-2)}{6}$ that satisfy planarity and maximality requirements. For more details on graph theory terminology, we refer to Nishizeki and Chiba [25].

In the literature, there are a few exact procedures for the WMPG problem and they can only solve small-sized instances. Foulds and Robinson [26] proposed a branch-and-bound (B&B) tree-search algorithm based on complete enumeration with planarity testing. Instances of 10 vertices were solved. Hasan and Osman [27] reported a B&B tree-search procedure using a Lagrangean relaxation bound, solving instances of sizes up to 17 vertices. Junger and Mutzel [5,6] designed an exact branch-and-cut algorithm with planarity testing, solving instances of sizes up to 10 vertices on a complete graph and up to 20 vertices on a sparse graph consisting of only 20 percent of all edges. Recently, Osman et al. [28] introduced a new integer linear-programming (LP) formulation and proposed LP-based meta-heuristics.

However, there are a large number of approximate algorithms for the WMPG problem. These can be grouped into *classical* heuristics and recent *meta-heuristics*. The classical heuristics can be grouped as follows. *Construction heuristics with planarity testing* include Foulds et al. [29] and Osman and Hasan [30]. *Construction heuristics without planarity testing* include Foulds and Robinson [24], Eades et al. [31], Green and Al-Hakim [32], Leung [33], Boswell [34] and Wascher and Merker [35]. *Local search improvement heuristics* include Foulds et al. [29], Al-Hakim [36], Kim and Kim [37], Hasan and Osman [27], and Pesch et al. [38]. However, there is a little published research on meta-heuristics and they include the hybrid simulated annealing and TS approach of Hasan and Osman [27], the linear-programming based meta-heuristics of Osman et al. [28] and the greedy random adaptive search procedure of Osman, et al. [39]. For reviews on other solution methods, we refer to Hassan and Hogg [3], Wascher and Merker [35], Domschke and Krispen [1], and Foulds [2].

3. The deltahedron construction heuristic

Foulds and Robinson [24] developed a construction procedure called the deltahedron heuristic (FR), which avoids planarity testing, by ensuring that the vertices are added into existing faces in such a way that the resulting sub-graph is always planar. It provides a good WMPG solution in a simple way using a series of triangulations or insertions. The FR heuristic computes for each vertex, $n_i \in V$, the weight $W(n_i) = \sum_{k \in E_i} w_k$, where E_i is the set of edges incident to n_i . The vertices are arranged in the non-increasing order of their weights in an ordered list (NL). The top four vertices $\{a, b, c, d\}$ of NL are selected to construct a planar sub-graph. Each pair of the four vertices is then connected to obtain an initial deltahedron—a complete sub-graph on the four vertices with four triangular faces: f_{abc} , f_{abd} , f_{adc} and f_{bcd} . The next vertex (e) from NL is selected for evaluation in the current set of triangular faces and is inserted in the face where it has the highest evaluation, see Fig. 2. The selection, evaluation and insertion processes are repeated until all vertices in NL are inserted, at which point a complete WMPG solution is obtained with $3n - 6$ edges and $2n - 4$ triangles (or faces).

It should be noted that several extensions were presented to enhance the performance of the FR heuristic. For instance, Green and Al-Hakim [32] heuristic (GH) modified the FR heuristic by selecting a vertex for insertion that has the highest evaluation among all un-inserted vertices, instead of selecting the next one in the ordered list NL. Osman et al. [39] developed a special data structure for the GH heuristic to enhance its efficiency. The data structure stores the first and second best insertion values for each vertex. As a result, the time complexity of the GH heuristic with the data structure (GH&D) was reduced from $O(n^3)$ to $O(n^2)$.

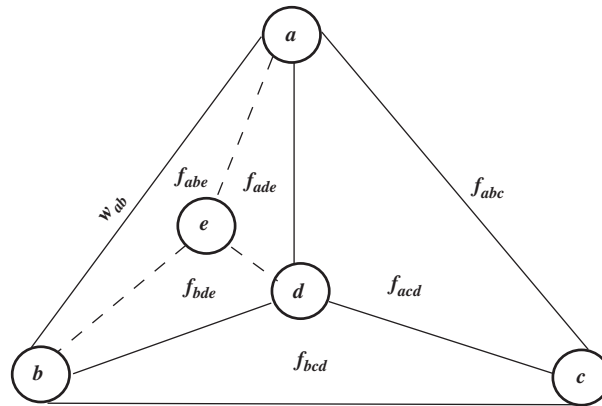


Fig. 2. Deltahedron with additional dashed edges after insertion of vertex e .

4. Tabu search implementation

Tabu search (TS) is a local-search based meta-heuristic that has been successfully applied to a widespread variety of combinatorial optimization problems. It was proposed by Glover [40]. In the present context, a basic TS approach guides any local search method to avoid getting trapped in local optima by iteratively moving from one WMPG solution G_p to another G'_p according to an aggressive *admission criterion*. The transition from G_p to G'_p is achieved by changing a set of edges, which are generated by a set of move operators. The admission criterion selects G'_p —the best improving or the least non-improving in the neighborhood of G_p —even if it causes deterioration in the objective value of G_p . To guard against the possibility of *cycling*, attributes of moves from recently visited solutions are memorized in a *tabu-list* and forbidden for a number of iterations, *tabu tenure* (tt). The forbidden attributes receive their status by reliance on an evolving memory, which allows this status to shift according to time or be overridden by an *aspiration criterion*. Finally, the TS algorithm terminates after a *stopping criterion* is met. TS applications for graph optimization problems include Hertz and de Werra [41], Osman and Christofides [42], Osman [43], Soriano and Gendreau [44], Blazewicz et al. [45], Osman and Wassan [46], Wassan and Osman [47]. We refer also to the webpage on tabu search by Rego and Glover [48]. The basic components of our TS implementation for the WMPG are explained next.

4.1. Representation of solution

Pesch et al. [38] developed a set of advanced moves to generate neighboring solutions. Data structures, based on a Star representation of $G_p = (V, E_p, W_p)$, were also introduced for representing solutions and managing moves. Given a vertex $n_i \in V$, let $\text{Star}(n_i)$ denote the list of vertices n_j such that the edge $\{n_i, n_j\}$ is in E_p . We say $\text{Star}(n_i)$ is facially ordered if for each two consecutive vertices n_r and n_s in this ordered list of vertices $\{\dots, n_r, n_s, \dots\}$, the triple of vertices $\langle n_i, n_r, n_s \rangle$ is a triangular face f_{irs} formed by $\{n_i, n_r\}$, $\{n_i, n_s\}$ and $\{n_r, n_s\}$ edges in E_p . By convention, it is said that the last vertex of $\text{Star}(n_i)$ immediately precedes the first one. The set of Stars for all $n_i \in V$ is denoted by $\text{Star}(G_p)$. We call $\text{Star}(G_p)$ uniformly ordered if every $\text{Star}(n_i)$ is ordered by assigning a common orientation (e.g., anti-clockwise) to the sequence in which the faces are encountered by this ordering. That means $\text{Star}(G_p)$

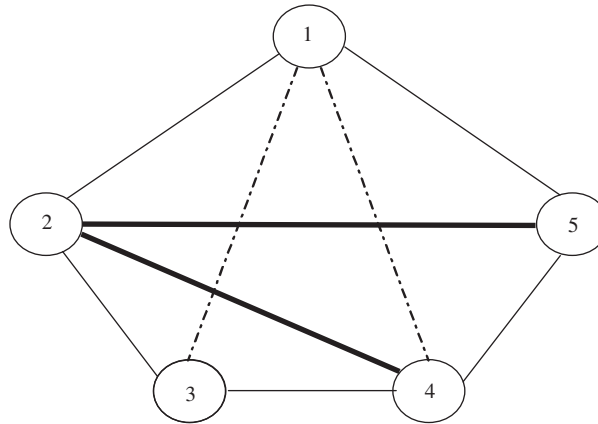


Fig. 3. A partner-shift move—edges a_{13} and a_{14} are replaced by edges a_{25} and a_{24} .

is uniformly ordered if there is a sequence of the faces of G_p such that the order of every $\text{Star}(n_i)$ is a subsequence. It was shown that $\text{Star}(G_p)$ and its associated data structures permit a highly efficient evaluation of candidate moves when transforming $\text{Star}(G_p)$ into another $\text{Star}(G'_p)$ by the advanced moves. Hence, the same solution representation and its structure will be used for our TS implementation and extended to handle two new additional move operators introduced here. For more details on the $\text{Star}(G)$ representation, we refer to the original paper of Pesch et al. [38].

4.2. Neighborhood generation mechanism

A neighborhood generation mechanism defines the set of move operators that transform a given solution, G_p , into another, G'_p . Let NEW be the set of move operators, then the set of all possible neighbors, G'_p , generated by NEW is called the neighborhood of G_p and is denoted by $N_{\text{new}}(G_p)$. For our implementation, NEW consists of a total of seven move operators. The first sub-set of move operators consists of the four advanced operators (AM) developed in Pesch et al. [38]. AM consists of a *partner-shift* move—a generalization of the diagonal move of Foulds et al. [29] that replaces two adjacent edges by two new ones, e.g., edges $\{1, 3\}$ and $\{1, 4\}$ by $\{2, 4\}$ and $\{2, 5\}$ in Fig. 3; and relocation of vertices moves of *degree-three*, *degree-four* or *degree-five* (see Figs. 4–6), where each relocation move removes either a vertex of degree three, four or five and re-inserts it in another place if that is deemed beneficial, while maintaining the planarity of the solution.

Figs. 3–6 depict the relocation moves generated by the advanced moves where the removed edges are marked in dashed lines and new edges are marked in solid ones. The basic principle behind the degree-four (or five) move is to reduce the degree of the relocated vertex from degree four (or five) to only degree-three. To maintain planarity, such moves require the addition of a set of either one edge (or two edges) to the changed part of G_p . For example, the additional set contains the best diagonal edge in the case of degree-four move (say edge $\{4, 6\}$ in Fig. 5), or the two edges forming the best partner-shift move in the case of degree-five move (say edges $\{5, 2\}$ and $\{5, 7\}$ in Fig. 6). As a result, the relocated vertex 5 in Fig. 5 (or 3 in Fig. 6) has a new degree of three and is then inserted in the best possible face of G_p .

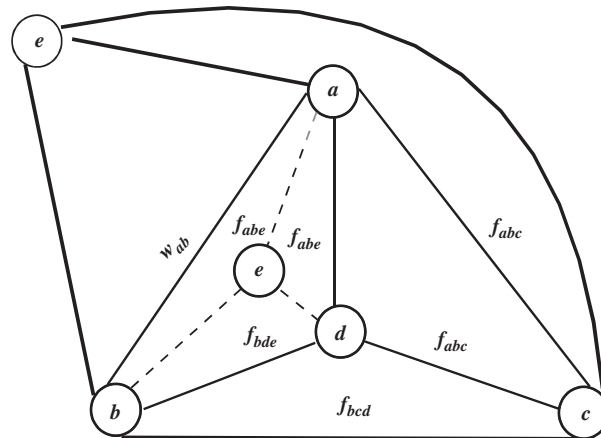


Fig. 4. A degree-3 relocation move—vertex e is removed from f_{abd} to external f_{abc} .

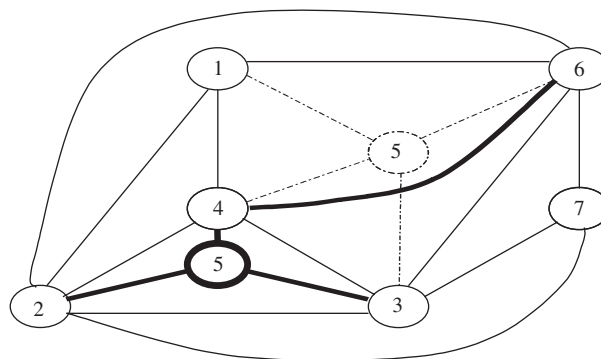


Fig. 5. A degree-4 relocation—vertex 5 from deltahedron $(1, 4, 3, 6)$ into face f_{234} .

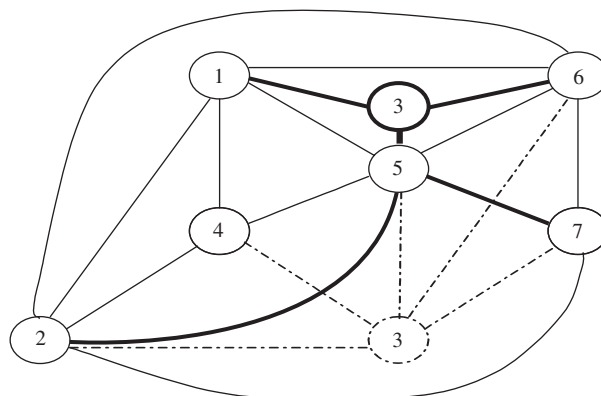


Fig. 6. A degree-5 relocation move—vertex 3 into a new face f_{156} .

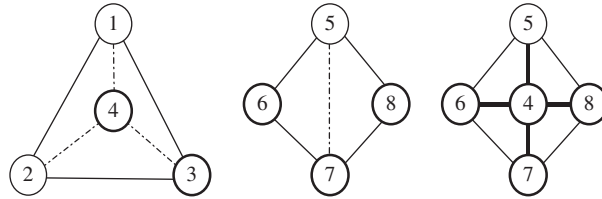
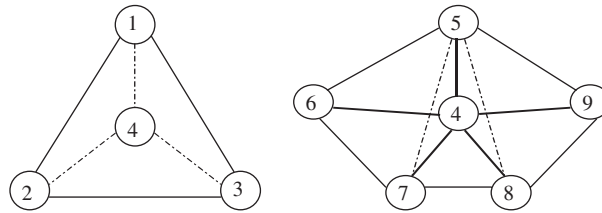


Fig. 7. A split-edge move—vertex 4 of degree 3 into edge {5, 7}.

Fig. 8. A split-face move—insertion of vertex 4 in the middle face f_{567} .

The fifth and sixth move operators are newly introduced in this work. The fifth move operator, which we call a *split-edge* move operator, attempts to insert a vertex of *degree three* on any existing edge. If the execution is beneficial, it will then change the degree of the involved vertex from *three to four*. For example, vertex 4 is splitting edge {5, 7} into two new edges {5, 4} and {4, 7} in Fig. 7. The sixth move operator, which we call a *split-face* move operator, attempts to insert a vertex of *degree three* in the middle of any *three consecutive faces*. If the execution is beneficial, it will change the degree of the involved vertex from three to five by destroying the three consecutive faces. For example, vertex 4 is relocated into in the middle face f_{578} between f_{567} and f_{589} faces in Fig. 8, resulting in removing the five dashed edges and adding the five bold edges.

Finally, the last move operator is the vertex re-labeling operator of Kim and Kim [37], which considers any two pair of vertices and swaps their positions, i.e., it keeps unchanged the connectivity structures of the swapped vertices, while changing only their labels to evaluate the corresponding move. This operator is a generalization of the swap operator move of Hasan and Osman [27] that restricts the swap to pairs of degree-three vertices in an 1–interchange mechanism.

4.3. Tabu restrictions

Tabu restrictions are imposed on the set of edges involved in the executed moves. Three criteria were considered. The first is based on the set of *edges dropped* from previous solutions, with the objective of preventing dropped edges from returning to any solution for a number of tabu-tenure (tt) iterations. The *second criterion* is based on the set of *edges added* to previous solutions, with the objective to force added edges to remain in the current solution for tt iterations. The last criterion uses simultaneously both the first and second criteria.

4.4. Tabu list and tenure

Our TS list takes the form of a TL matrix of $n \times n$ dimensions where the upper triangle indicates whether an edge is in E_p or not, while the lower triangle records the iteration number when an edge is added or dropped from G_p plus an additional tabu-tenure (tt) value. The tt value is found to be dependent on the number of vertices (n) and the existence/absence of a diversification strategy.

Assuming the vertices are labeled from 1 to n , the existence of an edge $\{i, j\} \in E_p$ can be easily identified from the 0–1 value in the upper entry, TL (i, j) for all $i < j$. Its corresponding tabu status can be checked by comparing the lower entry, TL (j, i), with the current iteration value. If the former is found to be smaller than the latter, then *the edge is called tabu*; otherwise it is *admissible*. A move is called *tabu* if it involves any tabu edge. Since the inclusion of some tabu edges may lead to improving moves, an *aspiration criterion*, that deems a tabu move admissible if the move leads to a new solution with a better objective value than the current best objective value, is implemented.

4.5. Move cache memory

Motivated by the application of cache memory in personal computers and its success in speeding up execution time, we have implemented a “move cache memory” to improve the performance of our TS algorithm. The computational effort of TS depends on the complexity of the move operators when exploring the search space. Such effort can be reduced significantly by the design of appropriate data structures to manage generated information and use it efficiently when evaluating future moves. The following ideas are developed for storing and updating the move-cache memory for fast exploration of the neighborhood search.

4.5.1. Move cache memory storage

In the same way that cache memory in computers holds information that the processor will need shortly for processing, the move cache memory holds information on the best move for each vertex in a WMPG solution for fast retrieval and evaluation. The move-cache memory (MCM) takes the form of a single matrix of $n \times 9$ dimensions. The rows correspond to the n vertices of the WMPG solution. The columns contain information about the characteristics of the best move associated with each vertex. For a given vertex, the following information is stored: its label n_i ; its current degree; its move operator; the list of connected vertices before the execution of the move, Star_B(n_i); the list of new connections after the execution, Star_A(n_i); special note on edges and/or vertices; the objective change, which is measured by the difference in weights between the new added and the old removed edges; the probability of selecting a vertex from MCM, which is based on the objective change to the total sum of changes for all non-tabu vertices; and finally its cumulative probability interval.

For illustration, Table 1 shows the move-cache memory for some typical moves for the WMPG solution given in Fig. 5. The non-tabu vertices are arranged in decreasing order of their objective changes (column 7) from best to worst. In a classical TS implementation, the move associated with vertex 5 in row one would be executed as it is the best non-tabu move in Table 1. Its corresponding information is then retrieved from the move cache matrix (Table 1) for implementation. For illustration, let us explain the meaning of the information contained in a typical row. Let us consider row one as an illustrative example of MCM information associated with the split-edge move of vertex 5 into edge $\{4, 6\}$ of Fig. 5. Note that the special note under column 6 contains different information for each vertex. Let assume that

Table 1

A typical move cache matrix for some moves in Fig. 5

n_i	Current degree	Move operator	Star_B	Star_A	Special note	Change value	Probability	Cumulative probability
5	3	Split	$\langle 4, 2, 3 \rangle$	$\langle 1, 4, 3, 6 \rangle$	$\langle 4, 6 \rangle$	+10	0.32	[0.68–1.0[
2	6	Swap	$\langle 7, 3, 5, 4, 1, 6 \rangle$	$\langle 6, 3, 2 \rangle$	$\langle 7 \rangle$	+5	0.26	[0.42–0.68[
1	3	Relocation	$\langle 2, 4, 6 \rangle$	$\langle 5, 2, 3 \rangle$	...	+4	0.25	[0.17–0.42[
4	5	Partner shift	$\langle 1, 2, 5, 3, 6 \rangle$	$\langle 1, 2, 5 \rangle$	$\langle 5, 6, 1 \rangle$	–2	0.17	[0.00–0.17[
7	3	Relocate	$\langle 6, 3, 2 \rangle$	$\langle 1, 4, 5 \rangle$	...	–15	0.00	[0.00–0.00[
3	5	Relocate	$\langle 7, 6, 4, 5, 2 \rangle$	$\langle 6, 1, 4 \rangle$	$\langle 7, 4, 5 \rangle$	...	0.00	[0.00–0.0[

the vertex associated with the best move in column one is $\{x\}$, then we would extract the following information from the “special note” column. If it contains only one vertex $\langle a \rangle$, then the vertex $\{a\}$ is to be swapped with $\{x\}$; if it has two vertices $\langle a, b \rangle$, then $\{a, b\}$ is to be split by vertex $\{x\}$ or $\{a, b\}$ is the single edge to be added if $\{x\}$ is associated with a degree-4 relocation move; and if it contains $\langle a, b, c \rangle$, then $\{x\}$ is associated with a degree-5 relocation move for which $\{a, b\}$ and $\{a, c\}$ are the two additional partner edges. Finally, the last row in the Table contains similar information about the best move of a tabu-vertex (say vertex 3), and hence no information on its probability selection is given. The tabu-vertex information may be used later if the tabu status of a corresponding vertex is removed and the move becomes valid. It should be noted that some of the elements of the matrix will be more clearly explained in the upcoming sections. For example, the last two columns are explained in details in Section (4.6.3).

4.5.2. Smart move cache memory update

Upon execution of a move to transform G_p into G'_p , not all parts of G_p change. Consequently, not all MCM information needs updating. The objective of smart updating is to focus only on the regions of G_p that have changed. Initially, the MCM table records information about the best move for each vertex. Upon execution of a move, the best moves associated with the subset of vertices determined from the lists Star_B and Star_A may change. For illustration, assume that the best move associated with vertex 5 in Table 1 is executed. The following changes can then be easily identified from MCM. Assuming an anti-clockwise orientation of vertices, the new faces $f_{5,6,1}$; $f_{5,1,4}$; $f_{5,4,3}$; $f_{5,3,6}$ and $f_{4,2,3}$ are identified from Star_A = $\langle 1, 4, 3, 6 \rangle$, and the removed faces $f_{5,4,2}$, $f_{5,2,3}$ and $f_{5,3,4}$ are identified from Star_B = $\langle 4, 2, 3 \rangle$. Moreover, the special note on the split-edge $\langle 4, 6 \rangle$ together with the other vertices from Star_A = $\langle 1, 4, 3, 6 \rangle$ identify the additional removed faces $f_{4,3,6}$ and $f_{6,1,4}$. Finally, Let $C = \text{Star_A} \cup \text{Star_B}$, each of the seven move operators used is then attempted on each vertex in C to update its best move in MCM.

4.5.3. MCM computational efficiency

The storage and update mechanisms implemented for MCM remove redundant computation in move evaluations. Hence, the extra savings in computation can be invested in searching further the neighborhood space for better solutions. Let us describe such savings on the degree-three move-relocation operator. After executing the split move associated with vertex 5 in Fig. 5, the set of degree-three vertices reduces from the triplet $\langle 5, 7, 1 \rangle$ to the singleton $\langle 7 \rangle$. Instead of inserting vertex 7 in all $2n - 4$ faces (10 faces

in Fig. 5), it is evaluated in only the new 5 changed faces and compared to its current best value in MCM. Hence, the complexity of degree-three move-relocation operator reduces from $O((2n - 4)\eta)$ to only $O(c\eta)$, where c is a constant equal to the number of *new* faces ($c < n$) and η is the current number of vertices of degree three. Similar reductions are achieved when the other move operators are attempted on the restricted regions. The overall reduction in computational effort is of an order of magnitude (around 700% on average) as will be shown empirically in the section on computational results.

4.6. Deterministic and probabilistic tabu search variants

In a deterministic TS (DTS), the best admissible move is aggressively selected for execution at the next iteration. However, the performance of DTS depends on factors such as the imperfect choice of admissible criteria, which are based on approximated tabu restrictions; the length of tabu tenure tt ; the ability of aspiration criterion to correct any wrong restrictions; the complexity of move operators and the required computational effort to evaluate moves. Although the set of seven move operators, and the MCM table and its management address some of the above-mentioned factors, DTS still has problems resulting from aggressive selection and the fixed length of tt .

In the following, a brief note on probabilistic TS (PTS) approaches in the literature is presented, followed by a presentation of our PTS variants: a random selection of moves (PTS1) from MCM and a random roulette selection of ranked moves from MCM (PTS2). Our variants are different from the literature in two aspects. First, our MCM is not a sample of randomly selected moves in the neighborhood, but contains the set of best admissible moves in the whole neighbourhood. Secondly, PTS1 randomly selects the next move for execution from the elite list of moves in MCM, whereas PTS2 selects the next move in a similar way to PTS1 but with an additional probabilistic bias.

4.6.1. A brief note on probabilistic tabu search

PTS approaches were developed to compensate for the existence of a level of noise that may exist when selecting moves. It is also used to diversify the search when it gets trapped in a local region and becomes unable to get out using the usual deterministic aggressive selection strategy. A PTS approach does not exclusively restrict the selection to the best move in the entire neighbourhood. Instead, it favors attractive candidates while still taking into account tabu restrictions and aspiration criteria. Variants include a restricted candidate list of evaluated moves with a random move acceptance approach, Glover [49], or a controlled randomisation approach based on a probability threshold for accepting a move [50]; a candidate list of moves which is efficiently generated from the entire neighborhood, with a simulated annealing acceptance criterion [43,51,42]; and a random candidate list of moves, arranged in non-increasing order of qualities, where the first candidate in the list is accepted as the next move with a constant probability, [45].

4.6.2. Random roulette selection of ranked moves

Our PTS implementation avoids getting trapped in local regions by alternating the search between intensification and diversification. At a given iteration of PTS, the best evaluation move is selected in the intensification phase. However, after a number of non-improving moves (NIM), the search enters the diversification phase for a number of diversification iterations (NDM), before returning again to the

intensification phase. Empirically, the number of iterations without improvement (NIM) was set to n iterations and the NDM to $n/10$.

When the search enters the diversification phase, the moves (with corresponding vertices) in MCM are ranked in non-increasing order of their change values. The next move is selected for execution by using a random roulette: a random value $0 \leq \theta \leq 1$ is chosen from the uniform distribution, and the vertex x with a cumulative probability interval containing θ is chosen. For example, if $\theta = 0.40$, then the partner-move associated with vertex 1 will be selected for execution as $\theta \in [0.17-0.42[$ in Table 1.

4.6.3. Probability and cumulative probability measures

The MCM table stores in Column 7 the change in the objective value of the best move for each vertex. We generate a probability value for each move using the following steps. First, the non-tabu moves are ranked in non-increasing order of their values and the tabu moves are listed in the last rows. Second, due to the existence of some non-improving moves with negative values, such as vertex 7 with a value (-15) in Table 1, the changes in the objective values are scaled to make them all positive. This scaling is achieved by adding the absolute value of the most negative value to the remaining objective changes. Third, the probability of each move is computed as the ratio of the scaled value over the total sum of all scaled values. For instance, the probability value (0.32) associated with the best move of vertex 5 is computed by dividing the new-scaled value $25 = (10 + |-15|)$ divided by the total sum of values (77). Last, after obtaining the probability value for each move, its cumulative probability interval is identified as follows. For instance, the highest evaluation move in Table 1 has a probability value of 0.32, its corresponding interval $[0.68-1.0[$ has a width equals to 0.32, then the second best move has an interval $[0.42-0.68[$ of width equal to its probability of selection 0.26 and so on until the worst and non-tabu moves are reached with interval widths of zero values.

4.6.4. Random selection of best moves

PTS1 selects moves randomly for diversification. This strategy is reasonable to investigate for two main reasons. First, the next move is randomly chosen from the list of non-tabu best moves in MCM and is not completely sampled. Random selection would not deteriorate the quality of the current solution much, but may provide a good escape strategy to nearby regions. Secondly, access time to choose a move from MCM is always constant.

4.7. The probabilistic diversification tabu search algorithm

Initialization

- Initialize TL (tabu list), MCM (move cache memory) and solution matrices to zeros.
- Set NIM—the number of non-improving iterations—to n .
- Set NDM—the number of diversification iterations—to $n/10$.
- Set tt —tabu tenure—to a value in the range $3n/8 \leq tt \leq 2 \times n$.
- Set MO—move operators—to the list of seven move operators—NEW.
- Set Max_Iter —the maximum number of iterations to stop—to $250 \times n$.
- Set tabu restrictions on the set of dropped edges.

Main execution

- Generate an initial solution G_p using the Deltahedron heuristic.
 - Scan $M_{\text{new}}(G_p)$ to identify the best admissible move for each vertex and store them in MCM table.
 - **While** {stopping criterion is not met} **Do**
 - {**Selection process**}
 - *If* (Iterations-Since-Last-Improvement > NIM) *Then*
 {**Perform a total of NDM diversification iterations**}:
 - Either, use the random roulette of ranked move strategy to select the next move from MCM for PTS2 implementation.
 - OR, use the random strategy to select the next move from MCM for PTS1 implementation.
 - *Else* {**Intensification**}:
 - Use the usual aggressive strategy to select the best move from MCM for execution;
 - *Endif*
 - {**Update process**}
 - Update the best, current solutions and associated records;
 - Update TL information;
 - Update the affected regions of MCM.
 - **EndWhile**
 - Report the best-found tabu solution, the total CPU time and other details.
- End of Main.**

5. Computational experience

A set of 100 test instances, generated by Osman et al. [28] and used in Osman et al. [39], was used to evaluate the performance of the WMPG algorithm. This set was generated according to edge weights specified in Foulds et al. [29]. For each size $n = 20, 40, 60, 80$ and 100 , complete graphs were generated with edge weights taken from a normal distribution with a mean of 100 and a standard deviation $\sigma = 5, 10, 20$ and 30 . For each pair of (n, σ) values, five instances (each using a different random generator seed) were generated. The proposed TS variants are evaluated in terms of solution quality and computational time. For each instance, the well-known classical upper bound (Z_{CUB}) is computed by adding up the weights of its highest $(3n - 6)$ edges. The relative percentage deviation (RPD) of the objective value (Z_A) of an algorithm “A” from Z_{CUB} , its relative percentage improvement (RPI) from the FR objective value (Z_{FR}) and the PTS2 objective value (Z_{PTS2}) are computed as follows:

$$\text{RPD} = \left(\frac{Z_{\text{CUB}} - Z_A}{Z_{\text{CUB}}} \right) \times 100 \quad \text{and} \quad \text{RPI} = \left(\frac{Z_{\text{FR}} - Z_A}{Z_{\text{FR}} - Z_{\text{PTS2}}} \right) \times 100.$$

For each pair of values of n and σ , the average of RPD (ARPD), the average of RPI (ARPI) and the average CPU time (ACPU) over five instances are reported. The averages of the performance are combined to produce a single new performance measure in Osman [52]. The new measure is called the Marginal Improvement per CPU unit of time, MIC, and it is applied for evaluating other algorithms in [53,54,58,59]. For a given heuristic, its MIC value is computed as the ratio of its ARPI over ACPU values. All the algorithms were coded in FORTRAN 99 and compiled using the Microsoft

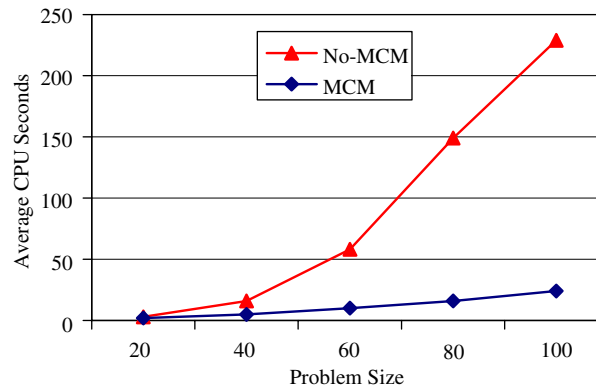


Fig. 9. Average CPU time in seconds for the move cache memory and no-MCM.

FORTRAN Power-Station 4.0. The experimental investigation was carried out using a personal computer with a Pentium™ Intel® Processor running 200 MHz with MMX Technology. The CPU has a 32 K internal cache and a total of RAM of 32 MB. The computational results from the literature are taken from Osman et al. [39].

5.1. Effect of MCM data structure

The average CPU time in seconds for each set of 20 instances of a given size n is presented in Fig. 9 for the PTS2 meta-heuristic. It is clear that the PTS2 variant, which uses the MCM data structure, requires far less computation time than the No-MCM variant, which does not use any data structure. For example, the average CPU times dropped from 91 seconds for No-MCM to 11.4 seconds with MCM. The ARPD of the savings in CPU seconds increases with the size of the instance. It varies from 50% to 854% for $n = 20$ –100, with an average savings of 698%. The savings illustrates the importance of MCM and the efficiency of our TS algorithm.

5.2. Effect of various components on TS

The performance of any meta-heuristic depends on its various components and their parameters settings. We shall investigate the effect of tabu restrictions, deterministic vs. probabilistic diversification strategies, initial solutions, and the new insertion moves on the performance of TS.

5.2.1. Effect of tabu restrictions

In Section 4.3, we discussed three possible criteria for tabu restrictions based on added edges, dropped edges or both added/dropped edges. In Fig. 10, the ARPD of solutions over all instances are plotted for different standard deviations, from $\sigma = 5$ to $\sigma = 30$. It can be seen that the tabu restriction based on added edges is the least effective, while the other criteria have similar performance. Since the third criterion requires more bookkeeping, the tabu restriction based on dropped edges is recommended for implementation.

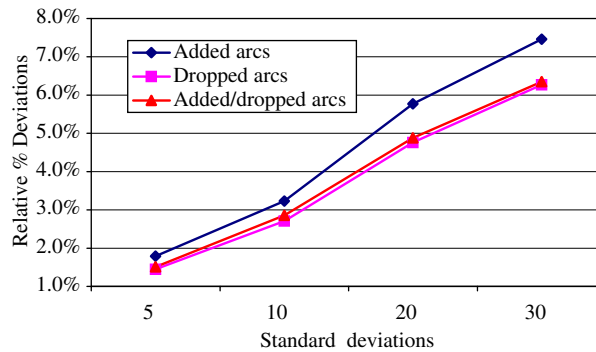


Fig. 10. Effect of tabu restrictions.

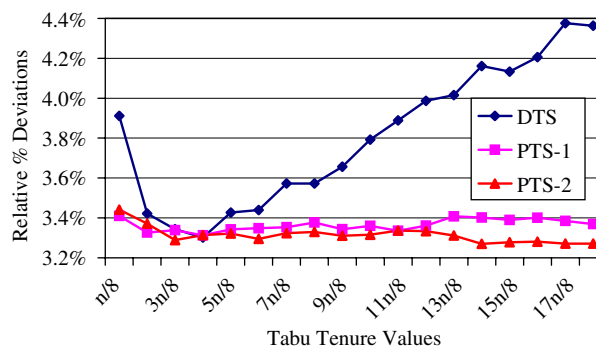


Fig. 11. Probabilistic vs. deterministic tabu search.

5.2.2. Deterministic vs. probabilistic strategies

To investigate the effect of diversification strategy on the quality of TS solution, we have considered the set of 20 instances of size $n = 40$ and executed each of the three TS variants: deterministic tabu search, DTS; random selection of moves from MCM for diversification, PTS1; and random roulette selection based on ranking of best moves in MCM for diversification, PTS2. Fig. 11 depicts the corresponding TS results under 18 different tt values, $tt = i * n/8$ for $i = 1, \dots, 18$. The quality of the DTS solution is found to be heavily dependent on the choice of the tt value. Any deviation from a good choice (here $i = 4$) would lead to a huge deterioration in its ARPD values. However, the probabilistic variants are more stable and less sensitive to the choice of tt value. Any reasonable choice would produce good solutions as can be seen in the figure. Finally, the ARPD values for PTS2 are always better than that of DTS and PTS1 variants; hence PTS2 is selected for future investigation.

5.2.3. Effect of initial solution on PTS2 performance

To investigate the effect of initial solution on PTS2 performance, two PTS2 variants are implemented. The first starts from the FR heuristic of Foulds et al. and the second starts from an improved FR heuristic (IFR). The IFR construction heuristic uses the same principle of FR heuristic for starting and ranking the vertices. However, IFR adds to the FR original insertion of a candidate vertex in each face, its insertion on each edge (the split-edge move) and its insertion in each middle face (the split-face move). The candidate

Table 2
ARPD of TS variants for different two initial different solutions

(n, σ)	Deltahedron (FR)			Improved deltahedron (IFR)			Best known	
	DTS	PTS1	PTS2	DTS	PTS1	PTS2	NEW	OLD
(20, 5)	0.78	0.72	0.73	0.74	0.72	0.72	0.72	0.797
(20, 10)	1.65	1.54	1.55	1.66	1.56	1.54	1.53	1.653
(20, 20)	2.93	2.83	2.83	2.93	2.86	2.80	2.79	2.945
(20, 30)	3.38	3.20	3.14	3.39	3.21	3.17	3.14	3.282
(40, 5)	1.40	1.35	1.30	1.45	1.32	1.34	1.27	1.501
(40, 10)	2.43	2.28	2.25	2.46	2.25	2.25	2.21	2.626
(40, 20)	4.49	4.21	4.14	4.59	4.19	4.20	4.08	4.571
(40, 30)	5.98	5.65	5.57	6.15	5.81	5.50	5.60	6.231
(60, 5)	1.73	1.56	1.56	1.70	1.54	1.53	1.54	1.765
(60, 10)	3.19	3.02	2.90	3.21	3.05	2.95	2.92	3.345
(60, 20)	5.47	5.00	5.00	5.53	5.05	4.99	4.83	5.717
(60, 30)	7.54	6.98	6.80	7.39	6.90	6.65	6.57	7.577
(80, 5)	1.93	1.76	1.74	1.96	1.80	1.71	1.71	2.003
(80, 10)	3.61	3.37	3.37	3.66	3.41	3.33	3.29	3.754
(80, 20)	6.14	5.68	5.62	6.20	5.75	5.64	5.50	6.431
(80, 30)	8.29	8.02	7.55	8.60	7.79	7.71	7.57	8.707
(100, 5)	2.17	2.03	1.99	2.13	2.01	1.99	1.94	2.258
(100, 10)	3.99	3.67	3.56	4.01	3.76	3.55	3.55	4.110
(100, 20)	6.95	6.39	6.30	6.98	6.35	6.33	6.16	7.211
(100, 30)	9.11	8.30	8.15	8.97	8.46	8.16	8.01	9.305
Average	4.16	3.88	3.80	4.19	3.89	3.80	3.75	4.289

vertex is then inserted in the best place associated with the best evaluation among the three types of insertions. The three tabu variants DTS, PTS1 and PTS2 are then executed on each instance for $250 \cdot n$ iterations, with a tabu restriction on dropped edges, and a tabu tenure $tt = n$. ARPD values for each pair of (n, σ) are reported in Table 2. The following observations can be made from the results. First, PTS2 is always better than PTS1, regardless of the starting initial solutions generated by either FR or IFR construction heuristics. Second, both probabilistic variants are better than the deterministic one. Third, it seems that the quality of TS is more dependent on the choice of move operators, appropriate diversification and settings of TS parameters than the choice of initial solution. On the contrary, initial solutions using the IFR construction heuristic did not produce better TS results than those obtained by the original FR construction heuristic. Finally, the last two columns show the old and new best known ARPD values. It is very clear the new ARPD values are significantly better than the old ones.

5.2.4. Effect of the new moves on the initial solution and the local search quality

Two classical heuristics are investigated in Table 3. The first heuristic, FR-AM, starts from a FR construction solution followed by a local search descent approach using a best improvement-selection strategy to explore the neighborhoods generated by the advanced moves (AM) of Pesch et al. [38]. The second IFR-NEW heuristic starts from the improved FR heuristic (IFR) solution followed by a local search

Table 3
Comparison of PTS2 with other heuristics in terms of ARPD values

Size (n, σ)	Meta-heuristics			Classical heuristics				
	PTS2	GRASP	LPI-PR	FR-AM	IFR-NEW	GH&D	FR	L
(20, 5)	0.72	0.853	1.046	1.007	1.017	1.265	1.472	1.161
(20, 10)	1.53	1.826	1.945	2.151	2.215	2.701	2.919	2.563
(20, 20)	2.79	3.076	3.380	3.904	3.575	4.891	5.051	4.077
(20, 30)	3.14	3.541	4.443	5.234	4.440	6.550	6.830	5.559
(40, 5)	1.27	1.548	1.643	1.808	1.666	2.258	2.257	1.697
(40, 10)	2.22	2.594	2.775	2.901	2.966	3.559	3.878	3.111
(40, 20)	4.08	4.670	5.580	5.595	5.236	6.533	7.321	5.746
(40, 30)	5.60	6.202	7.408	7.447	7.527	8.877	9.153	7.218
(60, 5)	1.54	1.754	2.009	1.918	1.978	2.267	2.555	1.946
(60, 10)	2.92	3.340	3.417	3.632	3.501	4.144	4.553	3.812
(60, 20)	4.83	5.685	6.191	6.269	6.038	7.213	8.104	6.253
(60, 30)	6.71	7.791	8.614	8.782	7.962	9.893	10.217	8.767
(80, 5)	1.73	1.986	2.203	2.173	2.099	2.423	2.698	2.201
(80, 10)	3.34	3.813	4.142	4.097	3.852	4.685	4.939	4.124
(80, 20)	5.50	6.474	7.283	6.858	6.758	7.849	8.786	6.754
(80, 30)	7.57	8.850	9.850	9.377	9.316	10.58	11.728	9.544
(100, 5)	1.94	2.278	2.486	2.439	2.273	2.727	2.963	2.322
(100, 10)	3.59	4.169	4.543	4.398	4.332	4.969	5.388	4.298
(100, 20)	6.16	7.255	7.822	7.642	7.428	8.565	9.380	7.571
(100, 30)	8.01	9.338	9.822	9.891	9.718	11.18	12.137	9.784
Average	3.76	4.352	4.830	4.876	4.695	5.566	6.116	4.925
Rank	1	2	3	2	1	4	5	3

descent approach using a best improvement-selection strategy to explore the neighborhoods generated by AM and the additional split-edge, split-face and interchange type of moves. The results by IFR-NEW and FR-AM are presented in Table 3 along with that of the three other classical heuristics in the literature, namely: the implementation of Osman et al. [39] with a new data structure for Green and Al-Hakim [32], GH&D; the construction heuristic of Foulds and Robinson [24], FR; the construction heuristic of Leung [33], L. The corresponding computational requirements are provided in Table 4. The provided results show clearly the positive effect of the new type of moves on improving the quality of FR solutions. The ranking of FR among the classical approaches is changed from worst to best in terms of solution quality as implemented in IFR-NEW and from worst to third using MIC ranking where both solution and computation requirement are taken into account.

5.3. Comparison of TS with other meta-heuristics

Our TS meta-heuristic is compared to the best two meta-heuristics in terms of solution quality (Table 3) and in terms of computation requirements (Table 4). Both tables have the same legends for

Table 4
Comparison of PTS2 with other heuristics in terms of all measures

Size (<i>n</i>)	Meta-heuristics			Classical heuristics				
	PTS2 ^a	GRASP ^c	LPI-PR ^b	FR-AM ^a	IFR-NEM ^a	G&HD ^c	FR ^c	L ^c
20	2	19.4	1.75	0.01	0.04	0.002	0	0.07
40	5	100.21	14.22	0.05	0.27	0.01	0	2.94
60	10	252.96	33.6	0.12	1.01	0.032	0.005	27.25
80	16	501.15	100.17	0.23	0.26	0.098	0.007	173.05
100	24	846.51	187.72	0.34	5.45	0.208	0.012	526.07
ACPU	11.4	344.05	67.49	0.15	1.406	0.07	0.00	145.88
Mflop/s	32	8.92	86	32	32	8.92	8.92	8.92
Adj-ACPU	11.40	95.90	181.38	0.15	1.406	0.02	0.00	40.66
ARPD	3.76	4.352	4.83	4.876	4.695	5.566	6.116	4.925
ARPI	100.00	74.87	54.58	52.63	60.31	23.34	0.00	50.55
MIC	8.77	0.78	0.30	350.88	42.89	1196.39	0.00	1.24
MIC Rank	1	2	3	2	3	1	5	4

Mflop/s: Millions of floating-point operations completed per second.

Adj-ACPU: Adjusted CPU time to an equivalent average on Pentium MMX 200 MHz.

MIC: Marginal improvement per CPU second values.

Rank: Performance rank based on MIC measures.

^aCPU time on a Desktop Pentium MMX technology, 200 MHz, with 32 MB of RAM.

^bCPU time on a Toshiba Laptop—Pentium II, 400 MHz with 128 MB of RAM.

^cCPU time on a Toshiba Laptop—Pentium 75 MHz, 8 MB of RAM.

columns. They are from left to right as follows: (*n*, σ), the instance size; our probabilistic TS variant (PTS2), which is based on a random roulette for diversification, tabu restrictions on dropped edges, a tabu tenure $tt = n$, and a longer execution of $450 * n$ iterations; the best GRASP with 1000 iterations of Osman et al. [39], GRASP; the best LP-based meta-heuristic of Osman et al. [28], LPI-PR. The last two lines show the average of ARPD values and the ranking of the meta-heuristic and classical algorithms in terms of solution quality from best (1) to worst (3) for meta-heuristics and from best (1) to worst (5) for classical heuristics.

The computational results based on ARPD values indicate a number of points. First, PTS2 is better than the best meta-heuristics in the literature; namely, GRASP and LP-based meta-heuristic (LPI-PR). Last, the Kruskal–Wallis rank sum test of hypothesis of multi-sample tests for differences between medians of the compared algorithms was performed, Levine et al. [55]. The null hypothesis was rejected with p -value = 0.00000 i.e., the medians are significantly different. Consequently, the computational results obtained by the compared meta-heuristics and classical heuristics are significantly different, i.e., PTS2 is the state of the art algorithm in terms of average solution quality.

Last, it is difficult to compare the CPU time, as it is not only the speed that dictates the performance, but cache, main memory, size of instances and compilers also play a role. However, using the CPU benchmark report of Dongorra [56] can help in converting the speed of different machines in terms of a universal unit Mflop/s—the rate of execution of millions of floating-point operations completed per second—as follows. Table 4 reports the average CPU time in seconds (ACPU) for the compared algorithms in

Table 3 with their corresponding computer machines. Since they use different machines, the ACPU values are converted into equivalent ACPU values on MMX 200 MHz in the Adj-ACPU row. The ARPI values and ARPD values are also reported. The ranking of algorithms based on the MIC measure shows that PTS2 is first among the meta-heuristics. Moreover, the Osman et al. [39] implementation (GH&D) of Green and Al-Hakim with data structure is first among the classical heuristics due to its good improvement in solution quality over FR using a small addition of CPU time. Further, it should be noted that although IFR-NEW is ranked first among classical heuristics in terms of solution quality (Table 3), but its MIC rank becomes due third due its time requirements (Table 4). In summary, MIC measure provides a good way of to rank algorithms. It takes into account both the quality of solution and the computational effort and allows a trade-off to be made between solution quality and computational requirement.

6. Conclusion

We have described an efficient and effective tabu search implementation for the weighted maximal planar graph problem. Its main features include two new move operators based on split-edge and split-face; a rich neighborhood generated by a set of seven move operators; efficient search of the neighborhood space using a move-cache memory table of best admissible moves; a random diversification strategy based on probabilistic bias to favor best-evaluation moves. Extensive computational results indicate that our probabilistic tabu search implementations are consistently better than the best meta-heuristics in the literature for the problem in terms of the MIC measure, which is based on solution quality and computation requirements.

A number of observations may be made. First, the tabu tenure value and existence of diversification determine to a great extent the quality of the deterministic tabu search solution. Second, the choice of initial solution does not have any effect on the quality of the tabu search solution in the presence of rich move operators. Third, the existence of a proper probabilistic diversification strategy marginalized the rigorous choice of tabu tenure value, made it a simple choice within a reasonable range. Fourth, it was found that the random roulette selection of ranked best moves provides better TS results than the simple random selection of best moves for diversification. Fifth, our probabilistic implementation should be investigated for other graph problems and in particular the graph planarization problem. Finally, the relationship and connection of the proposed moves to the move operators in Al-Hakim [57] should be analyzed and compared in the future.

Acknowledgements

The support of the Lebanese National Council for Scientific Search (LNCRS) and the grant of our University Research Board for this research are gratefully acknowledged. The author would like to thank Mr. Nikos Kalafatis and Dr. Musbah Barake for their helps in providing some of the results and the referees for their helpful comments that improved the paper.

References

- [1] Domschke W, Krispin G. Location and layout planning: a survey. *OR-Spektrum* 1997;19:181–94.
- [2] Foulds LR. *Graph theory applications*. New York: Springer; 1992.
- [3] Hassan MMD, Hogg GL. A review of graph theory application to the facility layout problem. *Omega* 1987;15:291–300.

- [4] Resende MGC, Ribeiro CC. A GRASP for graph planarization. *Network* 1997;29:173–89.
- [5] Junger M, Mutzel P. Solving the maximum weighted planar subgraph problem by branch-and-cut. In: Rinaldi G, Wolsey L, editors. *Proceedings of the third conference on integer programming and combinatorial optimization (IPCO)*. 1993. p. 479–92.
- [6] Junger M, Mutzel P. Maximum weighted planar subgraphs and nice embeddings: practical layout tools. *Algorithmica* 1996;16:33–59.
- [7] Giffin JW, Foulds LR, Cameron DC. Drawing a block plan from a REL-chart with graph theory and microcomputer. *Computers and Industrial Engineering* 1986;10:109–15.
- [8] Hassan MMD, Hogg GL. On constructing a block layout by graph theory. *International Journal of Production Research* 1991;32:231–4.
- [9] Al-Hakim LA. A modified procedure for converting a dual graph to a block layout. *International Journal of Production Research* 1992;30:2467–76.
- [10] Irvine SA, RinsmaMelchert I. A new approach to the block layout problem. *International Journal of Production Research* 1997;35:2359–76.
- [11] Watson KH, Giffin JW. The vertex splitting algorithm for facilities layout. *International Journal of Production Research* 1997;35:2477–92.
- [12] Giffin JW. Graph theory techniques for facilities layout. PhD thesis, University of Canterbury, New Zealand, 1984.
- [13] Osman IH. An introduction to metaheuristics. In: Lawrence M, Wilsdon C, editors. *Operational research tutorial papers*. UK Hampshire: Stockton Press, Publication of the Operation Society; 1995. p. 92–122.
- [14] Osman IH, Kelly JP. Meta-heuristics: an overview. In: Osman IH, Kelly JP, editors. *Meta-heuristics theory and applications*. Boston: Kluwer Academic Publishers; 1996. p. 1–21.
- [15] Osman IH, Laporte G. Meta-heuristics: a bibliography. *Annals of Operations Research* 1996;63:513–628.
- [16] Osman IH, Kelly JP. *Meta-heuristics theory and applications*. Boston: Kluwer Academic Publishers; 1996.
- [17] Rayward-Smith V, Osman IH, Reeves CC, Smith G. *Modern heuristic search techniques*. Chichester: Wiley; 1996.
- [18] Voss S, Martello S, Osman IH, Roucairol C. *Advances and trends in local search paradigms for optimization*. Boston: Kluwer Academic Publishers; 1999.
- [19] Ribeiro CC, Hansen P. *Essays and surveys in metaheuristics*. Boston: Kluwer Academic Publishers; 2002.
- [20] Glover F, Kochenberger G. *Handbook of metaheuristics*. Boston: Kluwer Academic Publishers; 2003.
- [21] Resende MGC, de Sousa JP. *Metaheuristics: computer decision-making*. Boston: Kluwer Academic Publishers; 2003.
- [22] Glover F, Laguna M. *Tabu search*. Boston: Kluwer Academic Publishers; 1997.
- [23] Gendreau M. Recent advances in tabu search. In: Ribeiro CC, Hansen P, editors. *Essays and surveys in metaheuristics*. Boston: Kluwer Academic Publishers; 2001. p. 369–77.
- [24] Foulds LR, Robinson DF. Graph heuristics for the plant layout problem. *International Journal of Production Research* 1978;16:27–37.
- [25] Nishizeki T., Chiba N. *Planar graphs: theory and algorithms*, *Annals of Discrete Mathematics*, vol. 32, North Holland, Amsterdam, 1988.
- [26] Foulds LR, Robinson DF. A strategy for solving the plant layout. *Operations Research Quarterly* 1976;27:845–55.
- [27] Hasan M, Osman IH. Local search algorithms for the maximal planar layout problem. *International Transactions in Operational Research* 1995;2:89–106.
- [28] Osman IH, Hasan M, Abdullah A. Linear programming based meta-heuristics for the weighted maximal planar graph. *Journal of Operational Research Society* 2002;53:1142–9.
- [29] Foulds LR, Gibbons P, Giffin J. Facilities layout adjacency determination: an experimental comparison of three graph theoretic heuristics. *Operations Research* 1985;33:1091–106.
- [30] Osman IH, Hasan M. A layered matching-planarity testing heuristic for the machine layout planning. *Egyptian Computer Science Journal* 1997;19:1–17.
- [31] Eades P, Foulds LR, Giffin JW. An efficient heuristic for identifying a maximum weight planar subgraph. *Lecture notes in mathematics*, vol. 952. Berlin: Springer; 1982, p. 239–51.
- [32] Green R, Al-Hakim LA. A heuristic for facilities layout planning. *Omega* 1985;13:469–74.
- [33] Leung J. A new graph-theoretic heuristic for facility layout. *Management Science* 1992;38:595–605.
- [34] Boswell S. TESSA—A new greedy heuristics for facilities layout planning. *International Journal of Production Research* 1992;30:1957–68.

- [35] Wascher G, Merker J. A comparative evaluation of heuristics for the adjacency problem facility layout planning. *International Journal of Production Research* 1997;35:447–66.
- [36] Al-Hakim LA. Two graph theoretical procedures for an improved solution to the facilities layout problem. *International Journal of Production Research* 1991;29:1701–18.
- [37] Kim J-Y, Kim Y-D. Graph theoretic heuristics for unequal-sized facility layout problems. *Omega* 1995;23:391–401.
- [38] Pesch E, Bartsch F, Glover T, Salewski F, Osman IH. Efficient facility layout planning in a maximally planar graph model. *International Journal of Production Research* 1999;37:263–83.
- [39] Osman IH, Al-Ayoubi B, Barake MA. A greedy random adaptive search procedure for the weighted maximal planar graph problem. *Computers and Industrial Engineering* 2003;45:635–51.
- [40] Glover F. Future paths for integer programming and links to artificial intelligence. *Computers and Operation Research* 1986;5:533–49.
- [41] Hertz A, de Werra D. Using tabu search techniques for graph coloring. *Computing* 1987;39:345–51.
- [42] Osman IH, Christofides N. Capacitated clustering problems by hybrid simulated annealing and tabu search. *International Transactions in Operational Research* 1994;1:317–36.
- [43] Osman IH. Heuristics for the generalized assignment problem: simulated annealing and tabu search approaches. *OR Spektrum* 1995;17:211–25.
- [44] Soriano P, Gendreau M. Diversification strategies in tabu search algorithm for the maximum clique problem. *Annals of Operations Research* 1996;63:189–208.
- [45] Blazewicz J, Salvador AM, Walkowiak R. Tabu search for two-dimensional irregular cutting. In: Ribeiro CC, Hansen P, editors. *Essay and surveys in meta-heuristics*. Boston: Kluwer Academic Publishers. p. 101–28.
- [46] Osman IH, Wassan N. A reactive tabu search metaheuristic for the vehicle routing problem with backhauls. *Journal of Scheduling* 2002;5:265–85.
- [47] Wassan N, Osman IH. Tabu search for the mix fleet vehicle routing problem. *Journal of the Operational Research Society* 2002;53:768–82.
- [48] Rego C, Glover F. www.tabusearch.net/Tabu_Search/Graph_Optimization.
- [49] Glover F. Tabu search—part I. *ORSA Journal on Computing* 1989;1:190–206.
- [50] Lokketangen A, Glover F. Probabilistic move evaluation in tabu search for zero-one mixed integer programming problems. In: Osman IH, Kelly JP, editors. *Meta-heuristics theory and applications*. Boston: Kluwer Academic Publishers 1996; p. 467–87.
- [51] Osman IH. Metastrategy simulated annealing and tabu search algorithms for the vehicle routing problem. *Annals of Operations Research* 1993;41:421–51.
- [52] Osman IH. Metaheuristics: Models, Design and Analysis. In: Kozan E, editor. *Proceedings of the fifth Asia-Pacific industrial engineering and management systems conference 2004, & the seventh Asia-Pacific division meeting of the international foundation of production research*, Gold Coast, Australia; 2004. p. 1.2-1–16.
- [53] Ahmadi S, Osman IH. Greedy random adaptive memory programming search for the capacitated clustering problem. *European Journal of Operational Research* 2005;162:30–44.
- [54] Ahmadi S, Osman IH. Density based problem space for the capacitated clustering p-median problem. *Annals of Operations Research* 2004;131:21–43.
- [55] Levine DM, Stephan D, Krhbiel TC, Berenson ML. *Statistics for managers using Microsoft Excel*. New Jersey: Prentice-Hall; 2001.
- [56] Dongarra JJ. Performance of various computers using standard linear equation software. Technical report, Computer Science Department, University of Tennessee, USA; 2004. <http://www.netlib.org/benchmark/performance.ps>.
- [57] Al-Hakim L. A note on efficient facility layout planning in a maximally planar graph model. *International Journal of Production Research* 2001;39:1549–55.
- [58] Osman IH, Ahmadi S. Guided construction search meta-heuristics for the capacitated clustering problem. *Journal of OR Society* 2006, in press.
- [59] Osman IH, Al-Ayoubi B. MIC analysis for comparing meta-heuristics. *Proceedings of the 6th metaheuristics international conference*. University of Vienna, Vienna, Austria; 2005. p. 725–32.